

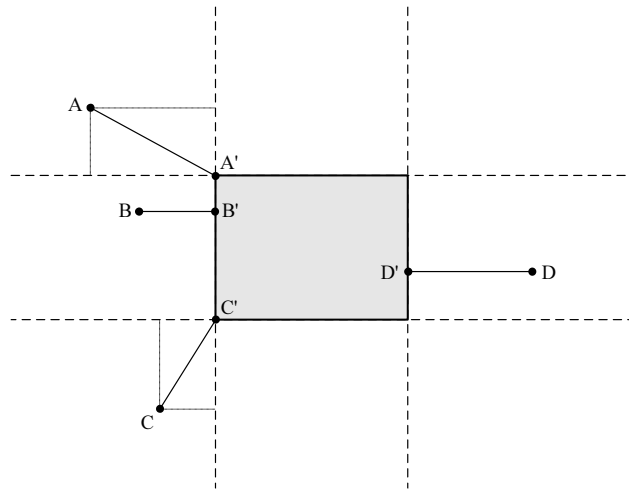


Figure 21.1.1. Distance from a point to a D -dimensional box. The general formula (as for lines AA' and CC') is a Pythagorean sum of D distances to the plane that includes the nearer side of the box. But when the point is between two such parallel planes (as for BB' and DD') then the corresponding coordinate is omitted from the sum.

inside. If inside-versus-outside, and not distance, is *all* you want to know, then some streamlining is possible: Replace `dd` by a Boolean variable, substitute logical-or's for the additions, and of course omit the square root. The logic remains otherwise the same.

21.1.2 Nodes for Binary Trees of Boxes

In the next section we will construct a *binary tree* of nested boxes, wherein each box is subdivided into, and linked to, two daughter boxes. Each box in the tree will also contain a list of points that lie inside the box. For use in such a task, we here give a structure, derived from the `Box` structure, but with additional variables that can point to a mother box, two daughter boxes, and the low and high indices on a list of points (designating the range of points inside the box). The constructor just sets all the values explicitly.

```
template<Int DIM> struct Boxnode : Box<DIM> {
Node in a binary tree of boxes containing points. See text for details.
    Int mom, dau1, dau2, ptlo, pthi;
    Boxnode() {}
    Boxnode(Point<DIM> mylo, Point<DIM> myhi, Int mymom, Int myd1,
            Int myd2, Int myptlo, Int mypthi) :
        Box<DIM>(mylo, myhi), mom(mymom), dau1(myd1), dau2(myd2),
        ptlo(myptlo), pthi(mypthi) {}
};
```

`kdtree.h`

21.2 KD Trees and Nearest-Neighbor Finding

Once, long ago, the term “kd tree” (or “k-D tree”) was an abbreviation for “ k -dimensional tree.” However, the term has come to mean a very specific, and

useful, kind of tree structure for partitioning points, especially in small numbers of dimensions, like 2 or 3. A KD tree that contains N points can be constructed in $O(N \log N)$ time and $O(N)$ space. Once constructed, the KD tree facilitates such operations as finding a point's nearest neighbor in $O(\log N)$ time, or all nearest neighbors in $O(N \log N)$ time. KD trees were first described by Bentley [8] in 1975. Let's see how this works.

Start with a very large box, one that easily contains all possible points that are of interest. There is no penalty for making this *root box* humongous, so coordinates of $\pm 10^{99}$ are fine. Now generate a list of N points (of interest to your application) that lie inside the root box. The defining principles of a KD tree are

- Boxes are successively partitioned into two daughter boxes.
- Each partition is along the axis of one coordinate.
- The coordinates are used cyclically in successive partitions.
- In making the partition, the position of the “cut” is chosen to leave equal numbers of points on the two sides (or differing by one in the case of an odd number of points).

Within these principles, there are some arbitrary design choices to be made. In the implementation below, the partition “cut” goes exactly through one of the points (i.e., shares one of its coordinate values). This avoids a bit of extra bookkeeping incurred by other possible choices. Also, we terminate the tree when a box node contains either one or two points, avoiding the additional partitioning of two-point boxes into two one-point boxes. This choice is natural because the *Boxnode* structure already has pointers to two points (*ptlo* and *pthi*), and it reduces the total number of stored boxes by as much as 50%.

With these principles and design rules in mind, you can decode Figure 21.2.1, which shows a two-dimensional KD tree containing 1000 points. (As a bit of artistic license, the root box in the figure has been shrunk to just contain the points, instead of being off near infinity.)

Interestingly, given N , the number of points, it is possible to give an exact formula for the number of boxes generated by our KD tree partition rules. (This makes memory allocation for the tree very straightforward.) If $N_B(N)$ is the number of boxes needed for N points, then two obvious recurrence relations describe what happens in the initial partitioning of $2n$ points into n plus n , or $2n - 1$ points into n plus $n - 1$:

$$\begin{aligned} N_B(2n) &= 2N_B(n) + 1 \\ N_B(2n - 1) &= N_B(n) + N_B(n - 1) + 1 \end{aligned} \quad (21.2.1)$$

The $+1$ in both formulas refers to the additional mother box that “glues together” two daughter partial trees at each stage. The solution to these recurrences is

$$N_B(N) = \min(M - 1, 2N - \frac{1}{2}M - 1) \quad (21.2.2)$$

where M is the smallest power of 2 greater or equal to N , that is,

$$M = 2^{\lceil \log_2 N \rceil} \quad (21.2.3)$$

(You can verify this solution by induction, working out the various possibilities of the min function. Or — much more fun — you can write a program to verify it numerically for $N < 10^9$, say.)

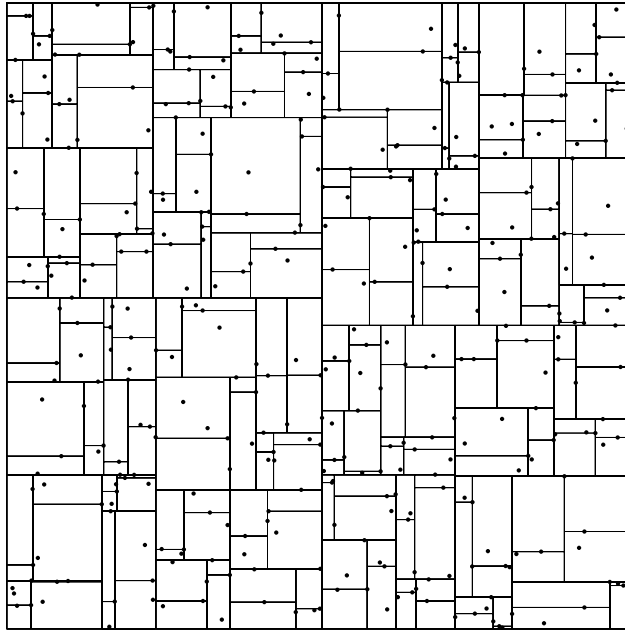


Figure 21.2.1. KD tree constructed from 1000 points in the plane. The first subdivision is visible as a full-height vertical line about halfway across the figure. The next subdivisions are horizontal lines, extending halfway across the figure. The subdivisions alternate between horizontal and vertical, and partition into (nearly) equal numbers of points at each stage. This tree terminates when there are either one or two points in a box (one of which is usually on the box boundary).

21.2.1 Implementation of the KD Tree

We implement the KD tree as a structure that gets built from a vector of `Points` and contains methods that embody the principal applications that we will discuss below, mainly various kinds of nearest-neighbor problems.

```
template<Int DIM> struct KDtree { kdtree.h
    Structure for implementing a KD tree.
    static const Doub BIG; Size of the root box, value set below.
    Int nboxes, npts; Number of boxes, number of points.
    vector< Point<DIM> > &ptss; Reference to the vector of points in the KD tree.
    Boxnode<DIM> *boxes; The array of Boxnodes that form the tree.
    VecInt ptindx, rptindx; Index of points (see text), and reverse index.
    Doub *coords; Point coordinates rearranged contiguously.
    KDtree(vector< Point<DIM> > &pts); Constructor.
    ~KDtree() {delete [] boxes;}
    Next, utility functions for use after the tree is constructed. See below.
    Doub disti(Int jpt, Int kpt);
    Int locate(Point<DIM> pt);
    Int locate(Int jpt);
    Next, applications that use the KD tree. See text.
    Int nearest(Int jpt);
    Int nearest(Point<DIM> pt);
    void nnearest(Int jpt, Int *nn, Doub *dn, Int n);
    static void sift_down(Doub *heap, Int *ndx, Int nn); Used by nnearest.
    Int locatenear(Point<DIM> pt, Doub r, Int *list, Int nmax);
};

template<Int DIM> const Doub KDtree<DIM>::BIG(1.0e99);
```

Note that the `KDtree` structure keeps a reference to the vector of Points that created it. This is used in some of the applications and has the implication that the user should not modify the vector of points while its derived KD tree is in scope. The array `coords` is an internal representation of the points-vector that is used during the construction of the KD tree, and then is immediately returned to the memory pool.

What makes the KD tree fast to construct is the existence of fast partition algorithms, $O(N)$ in time, that not only find the median value in an array of N values, but also move all smaller values to one side of the array and all larger values to the other. We already met such an algorithm in §8.5, in the routine `select`. Here, we need a slight variant, `selecti`, that partitions an array of integers not by their values, but by using them to index a separate array of values that remain unaltered. Because we will want to partition subsegments of arrays, we pass all references to the arrays by address.

```
kdtree.h  Int selecti(const Int k, Int *indx, Int n, Doub *arr)
Permutess indx[0..n-1] to make arr[indx[0..k-1]] ≤ arr[indx[k]] ≤ arr[indx[k+1..n-1]].
The array arr is not modified. See comments in the routine select.
{
    Int i,ia,ir,j,l,mid;
    Doub a;

    l=0;
    ir=n-1;
    for (;;) {
        if (ir <= l+1) {
            if (ir == l+1 && arr[indx[ir]] < arr[indx[l]])
                SWAP(indx[l],indx[ir]);
            return indx[k];
        } else {
            mid=(l+ir) >> 1;
            SWAP(indx[mid],indx[l+1]);
            if (arr[indx[l]] > arr[indx[ir]]) SWAP(indx[l],indx[ir]);
            if (arr[indx[l+1]] > arr[indx[ir]]) SWAP(indx[l+1],indx[ir]);
            if (arr[indx[l]] > arr[indx[l+1]]) SWAP(indx[l],indx[l+1]);
            i=l+1;
            j=ir;
            ia = indx[l+1];
            a=arr[ia];
            for (;;) {
                do i++; while (arr[indx[i]] < a);
                do j--; while (arr[indx[j]] > a);
                if (j < i) break;
                SWAP(indx[i],indx[j]);
            }
            indx[l+1]=indx[j];
            indx[j]=ia;
            if (j >= k) ir=j-1;
            if (j <= k) l=i;
        }
    }
}
```

The basic strategy for constructing the KD tree is this: Set up an array of integers that index the N points (`ptindx`, below). Next, copy all the point coordinates into an array (`coords`) in which all the x_0 coordinates are contiguous, followed by all the x_1 coordinates, and so on through all the dimensions. Now use `selecti` to partition (and rearrange) the index of points according to the value of their x_0 co-

ordinates, with half the points on each side of the partition. These two halves, now viewed as separate arrays, contain the points in two new daughter boxes. Now partition each of them into half by the value of the x_1 coordinate. And so on, recursively, going through the coordinates cyclically.

The recursion is so simple that it is easy to code it as a simple “pending task list,” thus avoiding the overhead of recursive function calls. A pending task consists of an index pointing to the box ready for further partitioning (the expectant mother, as it were) and a value that remembers which dimension is next to partition along. Because the tree is constructed “depth first,” the task list never grows larger than the log of the total number of boxes. Every new daughter box is born with a pointer to its mother, and pointers to its beginning and end elements in the points index array `ptindx`. Although these elements will generally be permuted in subsequent partitionings, none will ever be moved out of the range specified when a daughter box is first created. That is why the whole process can be done in a single point-index array, with all boxes simply pointing into some subrange of that array.

The KDtree constructor, below, should now be straightforward to understand.

```
template<Int DIM> KDtree<DIM>::KDtree(vector< Point<DIM> > &pts) :           kdtree.h
    ptss(pts), npts(pts.size()), ptindx(npts), rptindx(npts) {
Construct a KD tree from a vector of points.
    Int ntmp,m,k,kk,j,nowtask,jbox,np,tmom,tdim,ptlo,pthi;
    Int *hp;
    Doub *cp;
    Int taskmom[50], taskdim[50];           Enough stack for 250 points!
    for (k=0; k<npts; k++) ptindx[k] = k;   Initialize the index of points.
    Calculate the number of boxes and allocate memory for them.
    m = 1;
    for (ntmp = npts; ntmp; ntmp >>= 1) {
        m <= 1;
    }
    nboxes = 2*npts - (m >> 1);
    if (m < nboxes) nboxes = m;
    nboxes--;
    boxes = new Boxnode<DIM>[nboxes];
    Copy the point coordinates into a contiguous array.
    coords = new Doub[DIM*npts];
    for (j=0, kk=0; j<DIM; j++, kk += npts) {
        for (k=0; k<npts; k++) coords[kk+k] = pts[k].x[j];
    }
    Initialize the root box and put it on the task list for subdivision.
    Point<DIM> lo(-BIG,-BIG,-BIG), hi(BIG,BIG,BIG);           Syntax OK for 2-D too.
    boxes[0] = Boxnode<DIM>(lo, hi, 0, 0, 0, 0, npts-1);
    jbox = 0;
    taskmom[1] = 0;           Which box.
    taskdim[1] = 0;           Which dimension.
    nowtask = 1;
    while (nowtask) {           Main loop over pending tasks.
        tmom = taskmom[nowtask];
        tdim = taskdim[nowtask--];
        ptlo = boxes[tmom].ptlo;
        pthi = boxes[tmom].pthi;
        hp = &ptindx[ptlo];           Points to left end of subdivision.
        cp = &coords[tdim*npts];       Points to coordinate list for current dim.
        np = pthi - ptlo + 1;           Number of points in the subdivision.
        kk = (np-1)/2;                 Index of last point on left (boundary point).
        (void) selecti(kk, hp, np, cp); Here is where all the work is done.
        Now create the daughters and push them onto the task list if they need further subdividing.
```

```

    hi = boxes[tmom].hi;
    lo = boxes[tmom].lo;
    hi.x[tdim] = lo.x[tdim] = coords[tdim*npts + hp[kk]];
    boxes[++jbox] = Boxnode<DIM>(boxes[tmom].lo,hi,tmom,0,0,ptlo,ptlo+kk);
    boxes[++jbox] = Boxnode<DIM>(lo,boxes[tmom].hi,tmom,0,0,ptlo+kk+1,pthi);
    boxes[tmom].dau1 = jbox-1;
    boxes[tmom].dau2 = jbox;
    if (kk > 1) {
        taskmom[++nowtask] = jbox-1;
        taskdim[nowtask] = (tdim+1) % DIM;
    }
    if (np - kk > 3) {
        taskmom[++nowtask] = jbox;
        taskdim[nowtask] = (tdim+1) % DIM;
    }
}
for (j=0; j<npts; j++) rptindx[ptindx[j]] = j;      Create reverse index.
delete [] coords;                                  Don't need them anymore.
}

```

There are a small number of utility functions that are easy to provide. Although we generally prefer to have our distance (`dist`) functions be freestanding, it is useful to have a `KDtree` member routine that returns the distance between two points in a KD tree, referenced by their integer position in the underlying vector of points.

```

kdtree.h  template<Int DIM> Doub KDtree<DIM>::disti(Int jpt, Int kpt) {
Returns the distance between two points in the kdtree given their indices in the array of points,
but returns a large value if the points are identical.
    if (jpt == kpt) return BIG;
    else return dist(ptss[jpt], ptss[kpt]);
}

```

There is a special reason for returning `BIG` when the two points are identical: Later, when we find a point's nearest neighbor, we don't want the invariable answer to be "itself!"

Another simple function takes an arbitrary `Point` as the argument and returns the index of the box that uniquely contains it. In this function we first see an example of traversing the tree hierarchically, starting at the root box and then choosing only one of two daughter boxes at each stage. Also, by keeping track of which dimension is next to be partitioned on (`jdim`, below), we need only check one of the point's coordinates at each stage. Evidently, the whole process is $O(\log N)$ in time, since there can be only that many levels in the tree.

```

kdtree.h  template<Int DIM> Int KDtree<DIM>::locate(Point<DIM> pt) {
Given an arbitrary point pt, return the index of which kdtree box it is in.
    Int nb,d1,jdim;
    nb = jdim = 0;                                Start with the root box.
    while (boxes[nb].dau1) {                       As far as possible down the tree.
        d1 = boxes[nb].dau1;
        if (pt.x[jdim] <= boxes[d1].hi.x[jdim]) nb=d1;
        else nb=boxes[nb].dau2;
        jdim = ++jdim % DIM;                       Increment the dimension cyclically.
    }
    return nb;
}

```

The actual `Box` can be obtained from the returned integer, say `j`, by referencing `boxes[j]` in the `KDtree`, since `Boxnode` is a derived class of `Box`.

A very similar utility returns the index of the box that contains one of the points used to construct the KDtree. This is not necessarily the same box as the above routine would return, because of the possibility of multiple ties in coordinate values, in which case some tied points can lie on one side of the median partition and others on the other side.

```
template<Int DIM> Int KDtree<DIM>::locate(Int jpt) { kdtree.h
    Given the index of a point in the kdtree, return the index of which box it is in.
    Int nb,d1,jh;
    jh = rptindx[jpt]; The reverse index tells where the point lies in the
    nb = 0; index of points.
    while (boxes[nb].dau1) {
        d1 = boxes[nb].dau1;
        if (jh <= boxes[d1].pthi) nb=d1;
        else nb = boxes[nb].dau2;
    }
    return nb;
}
```

21.2.2 Applications of KD Trees

Most applications of KD trees make use of locality properties of its nested boxes. This is best seen in a few examples.

Suppose we want to know which of the N points in a KD tree is closest to an arbitrary point $\mathbf{p}$ (not necessarily one of the points in the tree). Without the tree, this is evidently a calculation that requires $O(N)$ operations, as we compare $\mathbf{p}$ to each candidate point in turn. However, if we have invested the $O(N \log N)$ operations required to construct the tree, then we can proceed in the following way. First, find the box in which $\mathbf{p}$ lies, and find the closest point in the tree that lies in that box. This takes $O(\log N)$ operations, as we saw above. The found point might in fact be the nearest neighbor (we don't know yet), but in any case its distance is now an upper bound on how far away the true nearest neighbor can be.

Second, traverse the tree by a depth-first recursion (exactly the way we did when we constructed the tree). As we encounter each new box, we check whether it could possibly contain a point closer than the nearest point found so far. Since we start with a point that is already pretty close (in the same box as $\mathbf{p}$), most boxes get rejected at this step. When a box is rejected, we *don't* need to open its daughter boxes, so a whole branch of the tree gets “pruned.” On average, only about $O(\log N)$ boxes actually get opened, so the total work load to find the nearest point is $O(\log N)$.

If we are really interested in only a single point $\mathbf{p}$, then the “slow,” $O(N)$, method would have been faster. But if we are repeating the operation for many different points $\mathbf{p}_i$, comparing to the same N points in the tree each time, then calling the following routine for each $\mathbf{p}_i$ in turn is a big win.

```
template<Int DIM> Int KDtree<DIM>::nearest(Point<DIM> pt) { kdtree.h
    Given an arbitrary location pt, return the index of the nearest point in the kdtree.
    Int i,k,nrst,ntask;
    Int task[50]; Stack for boxes waiting to be opened.
    Doub dnrst = BIG, d;
    First stage, we find the nearest kdtree point in same box as pt.
    k = locate(pt); Which box is pt in?
    for (i=boxes[k].ptlo; i<=boxes[k].pthi; i++) { Find nearest.
        d = dist(ptss[ptindx[i]],pt);
    }
```

```

    if (d < dnrst) {
        nrst = ptindx[i];
        dnrst = d;
    }
}
}
Second stage, we traverse the tree opening only possibly better boxes.
task[i] = 0;
ntask = 1;
while (ntask) {
    k = task[ntask--];
    if (dist(boxes[k],pt) < dnrst) {          Distance to closest point in box.
        if (boxes[k].dau1) {                If not an end node, put on task list.
            task[++ntask] = boxes[k].dau1;
            task[++ntask] = boxes[k].dau2;
        } else {                            Check the 1 or 2 points in the box.
            for (i=boxes[k].ptlo; i<=boxes[k].pthi; i++) {
                d = dist(ptss[ptindx[i]],pt);
                if (d < dnrst) {
                    nrst = ptindx[i];
                    dnrst = d;
                }
            }
        }
    }
}
}
return nrst;
}

```

What if we want to know the nearest-neighbor point not of an arbitrary location, but of one of the points stored in the KD tree? The above routine won't do. If we send it a point in the tree, it will give the obvious result that the point is its own nearest neighbor! We need to modify the routine so as to use `disti` from `KDtree`, which defined a point's self-distance as being large, rather than small.

An additional useful feature is to find not the single nearest neighbor, but the n nearest neighbors for some specified $n < N - 1$. The trick here is to avoid making the algorithm $O(n \log N)$, which is what it would be if, for each candidate point, we compared the candidate to all n of the best points so far. A good way to proceed is with a heap structure, as described in §8.3 and used (for a very similar purpose) in the routine `hpse1` in §8.5. The work load then scales as $O(\log n \log N)$.

The following routine is coded so as to lose hardly any efficiency in the case $n = 1$ (find the single nearest neighbor) while using a heap structure in the case $n > 1$.

```

kdtree.h  template<Int DIM> void KDtree<DIM>::nnearest(Int jpt, Int *nn, Doub *dn, Int n)
Given the index jpt of a point in a kdtree, return a list nn[0..n-1] of the indices of the n
points in the tree nearest to point j, and a list dd[0..n-1] of their distances.
{
    Int i,k,ntask,kp;
    Int task[50];          Stack for boxes to be opened.
    Doub d;
    if (n > npts-1) throw("too many neighbors requested");
    for (i=0; i<n; i++) dn[i] = BIG;
    Find smallest mother box with enough points to initialize the heap.
    kp = boxes[locate(jpt)].mom;
    while (boxes[kp].pthi - boxes[kp].ptlo < n) kp = boxes[kp].mom;
    Examine its points and save the n closest.
    for (i=boxes[kp].ptlo; i<=boxes[kp].pthi; i++) {
        if (jpt == ptindx[i]) continue;
        d = disti(ptindx[i],jpt);

```



```

    if (d < dn[0]) {
        dn[0] = d;
        nn[0] = ptindx[i];
        if (n>1) sift_down(dn,nn,n);    Maintain the heap structure.
    }
}

Now we traverse the tree opening only possibly better boxes.
task[1] = 0;
ntask = 1;
while (ntask) {
    k = task[ntask--];
    if (k == kp) continue;                Don't redo the box used to initialize.
    if (dist(boxes[k],ptss[jpt]) < dn[0]) {
        if (boxes[k].dau1) {                If not an end node, put on task list.
            task[++ntask] = boxes[k].dau1;
            task[++ntask] = boxes[k].dau2;
        } else {                            Check the 1 or 2 points in the box.
            for (i=boxes[k].ptlo; i<=boxes[k].pthi; i++) {
                d = disti(ptindx[i],jpt);
                if (d < dn[0]) {
                    dn[0] = d;
                    nn[0] = ptindx[i];
                    if (n>1) sift_down(dn,nn,n);    Maintain the heap.
                }
            }
        }
    }
}
}
return;
}

```

The following routine is used by the above for the sift-down process on the heap, differing from the `sift_down` used by `hpsort` (§8.3) only in its tailored interface for the present application, and the fact that it simultaneously rearranges two arrays, the distances (forming a heap) and the corresponding point numbers.

```

template<Int DIM> void KDtree<DIM>::sift_down(Doub *heap, Int *ndx, Int nn) {    kdtree.h
Fix heap[0..nn-1], whose first element (only) may be wrongly filed. Make a corresponding
permutation in ndx[0..nn-1]. The algorithm is identical to that used by sift_down in hpsort.
    Int n = nn - 1;
    Int j,jold,ia;
    Doub a;
    a = heap[0];
    ia = ndx[0];
    jold = 0;
    j = 1;
    while (j <= n) {
        if (j < n && heap[j] < heap[j+1]) j++;
        if (a >= heap[j]) break;
        heap[jold] = heap[j];
        ndx[jold] = ndx[j];
        jold = j;
        j = 2*j + 1;
    }
    heap[jold] = a;
    ndx[jold] = ia;
}

```

As a final illustrative example, here is how to find all points in a KD tree that lie within a specified radius r of some arbitrary location $\mathbf{p}$.

kdtree.h

```

template<Int DIM>
Int KDtree<DIM>::locatenear(Point<DIM> pt, Doub r, Int *list, Int nmax) {
    Given a point pt and radius r, returns a value nret such that list[0..nret-1] is a list of all
    kdtree points within a radius r of pt, up to a user-specified maximum of nmax points.
    Int k,i,nb,nbold,nret,ntask,jdim,d1,d2;
    Int task[50];
    nb = jdim = nret = 0;
    if (r < 0.0) throw("radius must be nonnegative");
    Find the smallest box that contains the "ball" of radius r.
    while (boxes[nb].dau1) {
        nbold = nb;
        d1 = boxes[nb].dau1;
        d2 = boxes[nb].dau2;
        Only need to check the dimension that divides the daughters.
        if (pt.x[jdim] + r <= boxes[d1].hi.x[jdim]) nb = d1;
        else if (pt.x[jdim] - r >= boxes[d2].lo.x[jdim]) nb = d2;
        jdim = ++jdim % DIM;
        if (nb == nbold) break;           Neither daughter encloses the ball.
    }
    Now traverse the tree below the starting box only as needed.
    task[1] = nb;
    ntask = 1;
    while (ntask) {
        k = task[ntask--];
        if (dist(boxes[k],pt) > r) continue;           Box and ball are disjoint.
        if (boxes[k].dau1) {                           Expand box further when possible.
            task[++ntask] = boxes[k].dau1;
            task[++ntask] = boxes[k].dau2;
        } else {                                       Otherwise process points in the box.
            for (i=boxes[k].ptlo; i<=boxes[k].pthi; i++) {
                if (dist(ptss[ptindx[i]],pt) <= r && nret < nmax)
                    list[nret++] = ptindx[i];
                if (nret == nmax) return nmax;           Not enough space!
            }
        }
    }
    return nret;
}

```

You might wonder why the above routine doesn't also use the tree structure to find cases where a box lies entirely inside the "ball" of radius r , in which case it could add the box's points to the output list without further opening of its daughters. The improvement is potentially a factor of $O(\log n)$, where n is the typical number of neighbors returned. The resulting routine is slightly too long for us to include, however. A good exercise is to code this modification yourself. You'll see that it is harder to check whether a box is inside a ball than vice versa: You have to check all 2^D corners of the box, not just the diagonally opposite "low" and "high" ones.

CITED REFERENCES AND FURTHER READING:

- Bentley, J.L. 1975, "Multidimensional Binary Search Trees Used for Associative Searching," *Communications of the ACM*, vol. 18, pp. 509–517.
- de Berg, M., van Kreveld, M., Overmars, M., and Schwarzkopf, O. 2000, *Computational Geometry: Algorithms and Applications*, 2nd revised ed. (Berlin: Springer), §5.2.
- Samet, H. 1990, *The Design and Analysis of Spatial Data Structures* (Reading, MA: Addison-Wesley).